API Contract v0.1 (draft)

What the ESP32 sends to the server, and what it gets back

Project	RFID + Fingerprint Attendance and Access Control
Sprint / Story	Sprint 1 — US_2003 (web development side)
Author	Akshit Saini
Date	1 September 2026
Version	0.1 — draft

1. Ground rules

- Base URL during development: http://<laptop-lan-ip>:3000 — for example http://192.168.1.24:3000
- Everything is JSON, both directions.
- Times are ISO 8601 in UTC, for example 2026-09-01T09:30:44Z.
- Every device request carries device_key. An unknown key gets HTTP 401.
- The device decides nothing. It reports what it read and obeys the answer.

This is marked v0.1 because I expect the hardware team to find things that need changing once they are on real hardware. It gets frozen after that.

2. POST /api/scan

One scan in, one decision out.

```
{
 "device_key": "DEV-MAIN-8F2A41C7",
 "card_uid": "04A2B7C1",
 "fingerprint_slot": 1,
 "scanned_at": "2026-09-01T09:30:44Z",
 "device_event_id": "MAIN-0011"
}
```

Field	Type	Required	Notes
device_key	string	yes	One per board, issued by me
card_uid	string	yes	Uppercase hex, no separators
fingerprint_slot	integer	no	Omit or null when no finger was presented
scanned_at	string	yes	Device clock, UTC
device_event_id	string	yes	Unique per device, never reused. Used for idempotency

Response, HTTP 200:

```
{
 "allowed": true,
 "direction": "IN",
 "person_name": "Akshit Saini",
 "reason": "OK",
 "event_id": 1041
}
```

A denial is still HTTP 200 — the request was valid, the person was not:

```
{ "allowed": false, "direction": "IN", "person_name": null, "reason": "FINGERPRINT_REQUIRED" }
```

Reason codes: OK, UNKNOWN_CARD, CREDENTIAL_EXPIRED, CREDENTIAL_REVOKED, NO_ACCESS_RULE, FINGERPRINT_REQUIRED, FINGERPRINT_MISMATCH, PERSON_INACTIVE, DEVICE_INACTIVE.

Errors: 400 for a malformed body, 401 for a bad device key, 500 for a server or database failure. On anything other than 200 the device should buffer the event and retry it through /api/sync.

3. POST /api/sync

Replay of events buffered while Wi-Fi was down. Up to 50 per call.

```
{
 "device_key": "DEV-MAIN-8F2A41C7",
 "events": [
  { "card_uid": "04A2B7C1", "fingerprint_slot": 1, "scanned_at": "...", "device_event_id": "MAIN-0012" }
 ]
}
```

```
{ "accepted": 1, "duplicates": 0, "rejected": 0, "results": [ ... ] }
```

Idempotent: an event whose device_event_id already exists for that device is reported as a duplicate and stored once. The device can safely re-send the whole batch after a timeout. Buffered events count for attendance, but the door has already closed on them — they do not open anything retrospectively.

4. POST /api/heartbeat

```
{ "device_key": "DEV-LAB-5C9E30B4", "firmware_ver": "0.1.0", "uptime_s": 8123 }
→ { "server_time": "2026-09-01T09:35:02Z" }
```

Updates last_seen_at so the dashboard can show a door as offline. The device can also use server_time to correct its own clock drift. Suggested interval: 60 seconds.

5. Reads for the web dashboard

Method	Path	Returns
GET	/api/occupancy	Who is inside right now, per room
GET	/api/events	Filtered audit log (from, to, person_id, room_id, limit)
GET	/api/people	People and their credentials
POST	/api/people	Add a person
POST	/api/credentials/:id/revoke	Revoke a lost card
GET	/api/health	Is the server up and is the database reachable

6. The decision, step by step

This is what the server does between receiving a scan and answering. The order matters, so the reason code that comes back is the real reason and not a side effect.

1. Is the device key known and active? If not, 401.
2. Does the card UID match a credential? If not, log UNKNOWN_CARD and deny.

3. Is the credential revoked, or past valid_until? Deny with the matching reason.
4. Is the person still active? If not, PERSON_INACTIVE.
5. Is there an access rule for this person, or their department, in this room? If not, NO_ACCESS_RULE.
6. If the rule requires a fingerprint: was one presented (else FINGERPRINT_REQUIRED), and does the slot belong to the card's owner (else FINGERPRINT_MISMATCH)?
7. Work out the direction: if the last allowed event for this person in this room was IN, this one is OUT, otherwise IN.
8. Insert the event — allowed or denied — and return the answer.

7. Open points with the hardware team

- Does the firmware event counter survive a reboot? device_event_id must never repeat.
- Is the ESP32 clock set from NTP, or should it rely on server_time from the heartbeat?
- How many events can the offline buffer hold before it overwrites?
- Should the relay pulse duration come back in the scan response, or stay in firmware?